

Design, Implementation and Evaluation of a Minimal HTTP/HTTPS Server

Performance comparison with nginx

Ernest Matskevich

Innopolis University

October 2026

Project Goal

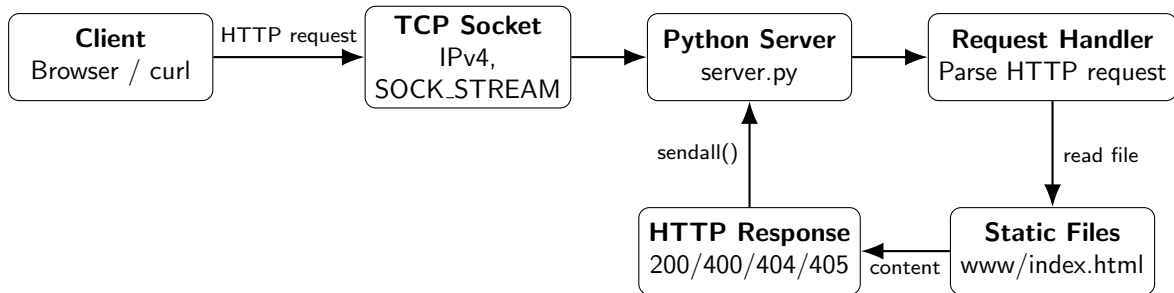
Main goal:

To design and implement a minimal HTTP/1.1 server directly on top of TCP sockets and experimentally evaluate its behavior, security, and performance.

Project objectives:

- Implement basic HTTP/1.1 request processing without web frameworks
- Serve static files and handle common HTTP status codes
- Support concurrent client connections
- Add HTTPS support using TLS and a server certificate
- Deploy the server in a Linux environment
- Analyze HTTP and HTTPS traffic using Wireshark
- Compare the custom server with nginx using performance benchmarks

System Architecture



Architecture

The server is implemented directly on top of TCP sockets. Each accepted connection is processed by a separate thread, while the HTTP protocol is parsed and generated by the application itself.

HTTP Request Processing

Raw request received from TCP socket

HTTP Request

```
GET /index.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.13.0
Accept: */*
```

TCP provides a stream of bytes. The server reads the data using:

```
client_socket.recv(4096)
```

The received bytes are decoded into text for HTTP processing.

Parsing the request

The first line is extracted:

```
GET /index.html HTTP/1.1
```

It is separated into three components:

Request Line

Method	GET
Path	/index.html
Version	HTTP/1.1

TCP bytes → HTTP text → Request line
→ Parsed fields

HTTP Response Handling

Request processing

After parsing the request, the server determines the appropriate response:

Supported Status Codes

200 OK

400 Bad Request

404 Not Found

405 Method Not Allowed

Generated HTTP response

The response is constructed manually and sent using:

```
client_socket.sendall(response)
```

Example Response

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html; charset=utf-8
```

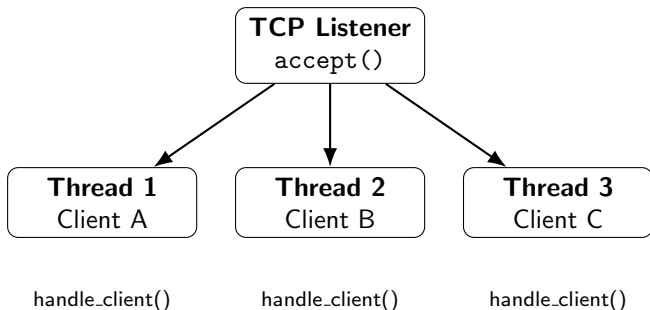
```
Content-Length: 198
```

```
Connection: close
```

```
<!DOCTYPE html>
```

```
<html> ... </html>
```

Concurrency Model



How it works

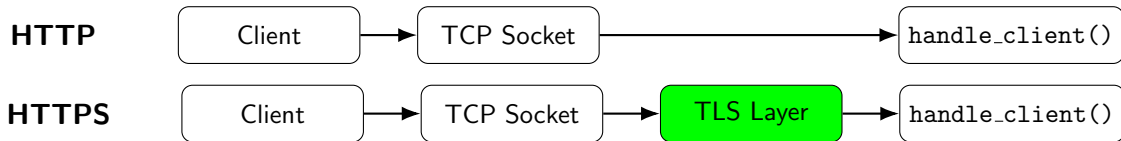
- 1 The listener waits for a TCP connection using `accept()`.
- 2 A new thread is created for each accepted client.
- 3 The thread executes `handle_client()`.
- 4 The listener immediately returns to `accept()`.

Concurrency Test

The `/slow` endpoint waits for 5 seconds.

While one thread was waiting, another client request was processed immediately by a different thread.

HTTPS / TLS Implementation



TLS Configuration

- Separate HTTPS listener on port 8443
- TLS context created with `ssl.SSLContext`
- Server certificate and private key are loaded
- TLS handshake is performed before HTTP processing

Socket Wrapping

Python

```
tls_socket = ssl_context.wrap_socket(
    client_socket,
    server_side=True
)
```

HTTP Traffic Analysis – Wireshark

```
> Frame 137: Packet, 122 bytes on wire (976 bits), 122 bytes captured (976 bits) on interface \Device\NPF_{Loopback}, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 11574, Dst Port: 8080, Seq: 1, Ack: 1, Len: 78
v Hypertext Transfer Protocol
  > GET / HTTP/1.1\r\n
    Host: localhost:8080\r\n
    User-Agent: curl/8.13.0\r\n
    Accept: */*\r\n
    \r\n
    [Response in frame: 139]
    [Full request URI: http://localhost:8080/]

0000  02 00 00 00 45 00 00 76 7b a5 40 00 80 06 00 00  ....E..v {:@....
0010  7f 00 00 01 7f 00 00 01 2d 36 1f 90 f6 0c c1 10  ....-6.....
0020  18 1f b0 2a 50 18 27 f9 19 0a 00 00 47 45 54 20  ...*P.'...GET
0030  2f 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74  / HTTP/1.1..Host
0040  3a 20 6c 6f 63 61 6c 68 6f 73 74 3a 38 30 38 30  : localhost:8080
0050  0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 63 75  ..User-Agent: cu
0060  72 6c 2f 38 2e 31 33 2e 30 0d 0a 41 63 63 65 70  rl/8.13.0..Accep
0070  74 3a 20 2a 2f 2a 0d 0a 0d 0a                    t: */*... ..
```

Observed:

- HTTP request is transmitted in plaintext
- Request method and path are directly visible
- HTTP headers can be inspected

HTTP Response – Unencrypted Content

```
> Frame 139: Packet, 349 bytes on wire (2792 bits), 349 bytes captured (2792 bits) on interface \Device\NPF_{Loopback}, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 8080, Dst Port: 11574, Seq: 1, Ack: 79, Len: 305
> Hypertext Transfer Protocol
v Line-based text data: text/html (10 lines)
  <!DOCTYPE html>\r\n
  <html>\r\n
  <head>\r\n
    <title>Minimal HTTP Server</title>\r\n
  </head>\r\n
  <body>\r\n
    <h1>Hello from my HTTP server!</h1>\r\n
    <p>This page is served by my own HTTP/1.1 server.</p>\r\n
  </body>\r\n
</html>
```

0020	f6 0c c1 5e 50 18 27 f9	eb db 00 00 48 54 54 50	...^P.' - ...HTTP
0030	2f 31 2e 31 20 32 30 30	20 4f 4b 0d 0a 43 6f 6e	/1.1 200 OK--Con
0040	74 65 6e 74 2d 54 79 70	65 3a 20 74 65 78 74 2f	tent-Type e: text/
0050	68 74 6d 6c 3b 20 63 68	61 72 73 65 74 3d 75 74	html; charset=ut
0060	66 2d 38 0d 0a 43 6f 6e	74 65 6e 74 2d 4c 65 6e	f-8--Con tent-Len
0070	67 74 68 3a 20 32 30 36	0d 0a 43 6f 6e 6e 65 63	gth: 206 --Connec
0080	74 69 6f 6e 3a 20 63 6c	6f 73 65 0d 0a 0d 0a 3c	tion: close....<
0090	21 44 4f 43 54 59 50 45	20 68 74 6d 6c 3e 0d 0a	!DOCTYPE html>..
00a0	3c 68 74 6d 6c 3e 0d 0a	3c 68 65 61 64 3e 0d 0a	<html>.. <head>..
00b0	20 20 20 20 3c 74 69 74	6c 65 3e 4d 69 6e 69 6d	<tit le>Minim
00c0	61 6c 20 48 54 54 50 20	53 65 72 76 65 72 3c 2f	al HTTP Server</
00d0	74 69 74 6c 65 3e 0d 0a	3c 2f 68 65 61 64 3e 0d	title>.. </head>..
00e0	0a 3c 62 6f 64 79 3e 0d	0a 20 20 20 20 3c 68 31	<body>.. <h1
00f0	3e 48 65 6c 6c 6f 20 66	72 6f 6d 20 6d 79 20 48	>Hello f rom my H
0100	54 54 50 20 73 65 72 76	65 72 21 3c 2f 68 31 3e	TTP serv er!</h1>

Visible in captured traffic:

- HTTP/1.1 200 OK
- Response headers
- Content-Type and Content-Length
- HTML document content

Security Problem

Anyone able to capture the traffic can read the HTTP request and response content.

HTTPS Traffic Analysis – TLS Encryption (1/3)

```
> Frame 97: Packet, 230 bytes on wire (1840 bits), 230 bytes captured (1840 bits) on interface \Device\NPF_{Loopback}, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 3618, Dst Port: 8443, Seq: 1, Ack: 1, Len: 186
√ Transport Layer Security
  [Stream index: 0]
  √ TLSv1.2 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 181
    > Handshake Protocol: Client Hello
```

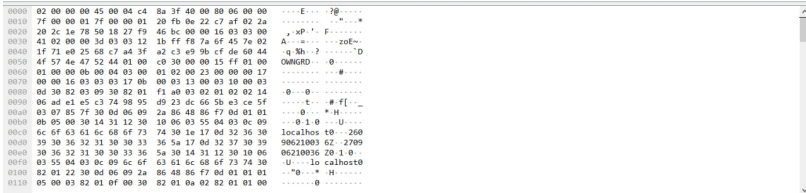
```
0000 02 00 00 00 45 00 00 e2 8a 3d 40 00 80 06 00 00 .....E...@.....
0010 7f 00 00 01 7f 00 00 01 0e 22 20 fb 20 2c 1d be ..... " . , ...
0020 c7 af 02 2a 50 18 27 f9 82 d3 00 00 16 03 03 00 ...*P.*.....
0030 b5 01 00 00 b1 03 03 6a 9f 04 e0 0d b2 82 c6 23 .....j.....#
0040 f5 11 c9 dc a7 24 41 24 e1 7b 11 ef 0f ce 22 62 .....$A$-{:....b
0050 ea 14 f8 1c c7 32 7b 00 00 2a c0 2c c0 2b c0 30 .....2{-.*.,+0
0060 c0 2f 00 9f 00 9e c0 24 c0 23 c0 28 c0 27 c0 0a ./.....$-#(-'..
0070 c0 09 c0 14 c0 13 00 9d 00 9c 00 3d 00 3c 00 35 .....-...<.5
0080 00 2f 00 0a 01 00 00 5e 00 00 00 0a 00 0c 00 00 ./.....^.....
0090 09 6c 6f 63 61 6c 68 6f 73 74 00 0a 00 08 00 06 ..localho st.....
00a0 00 1d 00 17 00 18 00 0b 00 02 01 00 00 0d 00 1a .....
00b0 00 18 08 04 08 05 08 06 04 01 05 01 02 01 04 03 .....
00c0 05 03 02 03 02 02 06 01 06 03 00 23 00 00 10 .....#.....
00d0 00 0b 00 09 08 68 74 7a 70 2f 31 2e 31 00 17 00 .....htt p/1.1...
00e0 00 ff 01 00 01 00 .....
```

Observed:

- TCP connection is still visible
- TLS handshake can be observed

HTTPS Traffic Analysis – TLS Encryption (2/3)

```
> Frame 99: Packet, 1224 bytes on wire (9792 bits), 1224 bytes captured (9792 bits) on interface \Device\NPF_{...}_id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 8443, Dst Port: 3618, Seq: 1, Ack: 187, Len: 1180
  > Transport Layer Security
    [Stream index: 0]
    > TLSv1.2 Record Layer: Handshake Protocol: Server Hello
    > TLSv1.2 Record Layer: Handshake Protocol: Certificate
    > TLSv1.2 Record Layer: Handshake Protocol: Server Key Exchange
    > TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done
```



```
0000  02 00 00 00 45 00 04 c4 8a 3f 40 00 00 06 00 00  ....E... ?@....
0010  7f 00 00 01 7f 00 00 01 20 fb 0e 22 c7 af 02 2a  ....xP...F.....
0020  20 2c 1e 78 59 18 27 f9 46 bc 00 00 16 03 03 00  ,.xP...F.....
0030  41 02 00 00 3d 03 03 12 1b ff f8 7a 6f 45 7a 02  A... ..zoE...
0040  1f 71 e0 25 68 c7 a4 3f a2 c3 e9 9b cf de 60 44  q 3h...? .....D
0050  4f 57 4e 47 52 44 01 00 c0 30 00 00 15 ff 01 00  OWNGRD...@.....
0060  01 00 00 0b 00 04 03 00 01 02 00 23 00 00 00 17  ....#.....
0070  00 00 16 03 03 03 17 0b 00 03 13 00 03 10 00 03  ....
0080  0d 30 82 03 09 30 82 01 f1 a0 03 02 01 02 02 14  .@...@...#...f[...
0090  06 ad e1 e5 c3 74 98 95 d9 23 dc 66 5b e3 ce 5f  ....t...#...f[...
00a0  03 07 85 7f 30 0d 06 09 2a 86 48 86 f7 0d 01 01  ...@...#...H.....
00b0  0b 05 00 30 14 31 12 30 10 06 03 55 04 03 0c 09  ...@...#...U.....
00c0  6c 6f 63 61 6c 68 6f 73 74 30 1e 17 0d 32 36 30  localhos t0...260
00d0  39 30 36 32 31 30 30 33 36 5a 17 0d 32 37 30 39  90621003 6Z...2709
00e0  30 36 32 31 30 30 33 36 5a 30 14 31 12 30 10 06  06210036 Z0...10...
00f0  03 55 04 03 0c 09 6c 6f 63 61 6c 68 6f 73 74 30  .U...lo calhost0
0100  82 01 22 30 0d 06 09 2a 86 48 86 f7 0d 01 01 01  .."0...#...H.....
0110  05 00 03 82 01 0f 00 30 82 01 0a 02 82 01 01 00  .....@.....
```

Observed:

- TCP connection is still visible
- TLS handshake can be observed
- HTTP request is no longer readable
- HTML response is no longer visible

HTTPS Traffic Analysis – TLS Encryption (3/3)

```
> Frame 105: Packet, 151 bytes on wire (1208 bits), 151 bytes captured (1208 bits) on interface \Device\NPF_{Loopback}, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 3618, Dst Port: 8443, Seq: 280, Ack: 1423, Len: 107
  Transport Layer Security
    [Stream index: 0]
    TLSv1.2 Record Layer: Application Data Protocol: Application Data
      Content Type: Application Data (23)
      Version: TLS 1.2 (0x0303)
      Length: 102
      Encrypted Application Data: 0000000000000000197c7b9fe8789048c200540efc932dcfe32d413ae35737d81c9cf1973e2338a39243a9e5da16a99696fbef6b60b20d710c289021066c9c07469...
```

```
0000 02 00 00 00 45 00 00 93 8a 45 10 00 00 06 00 00  ....E...E...
0010 7f 00 00 01 7f 00 00 01 0e 22 20 fb 20 2c 1e d5  .......*...
0020 c7 af 07 b8 50 18 27 f4 35 96 00 00 17 03 03 00  ....P...5....
0030 66 00 00 00 00 00 00 00 01 97 c7 b9 fe 87 89 04  f.....
0040 8c 20 05 40 ef c9 32 dc fe 32 d4 13 ae 35 73 7d  .@.-2.-2...5s}
0050 81 c9 cf 19 73 e2 33 8a 39 24 3a 9e 5d a1 6a 99  ...s-3-9$:-]j.
0060 69 6f be f6 b6 0b 20 d7 10 c2 89 02 10 66 c9 c0  io.....f...
0070 74 69 9e cb 1a 5d f2 82 18 a9 55 8a ad 8e c0 32  ti...]-U...2
0080 59 9f 41 c5 3c 2c 69 72 08 36 bf b3 0e a7 0e 12  Y-A...ir...6....
0090 e4 e6 9b 2c db 27 79  ....y
```

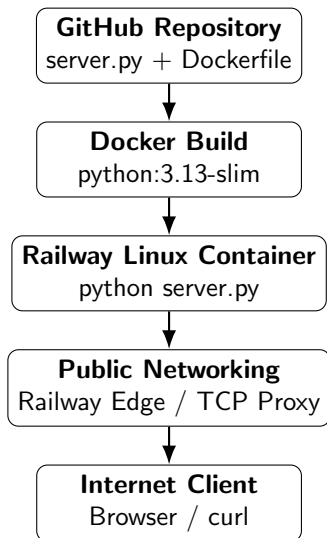
Observed:

- TCP connection is still visible
- TLS handshake can be observed
- HTTP request is no longer readable
- HTML response is no longer visible
- Application data is encrypted

Result

HTTP is transmitted inside the encrypted TLS channel.

Linux Deployment on Railway

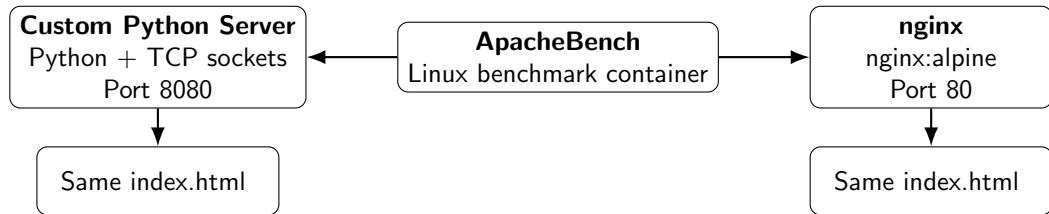


- Application runs inside a Linux Docker container
- Server binds to `0.0.0.0`
- HTTP port is provided through the `PORT` environment variable
- No external Python dependencies are required

Dockerfile

```
FROM python:3.13-slim
WORKDIR /app
COPY . .
CMD ["python", "server.py"]
```

Comparison with nginx



Controlled Conditions

- Both services run on Railway
- Both serve the same HTML document
- Tests use Railway private networking

ApacheBench Configuration

Identical workload

```
ab -n 1000 -c 10 <URL>
```

- 1000 requests per run
- Concurrency level: 10
- 3 runs for each server

Performance Benchmark Results

ApacheBench: 1000 requests, concurrency 10, 3 runs

Metric	Custom Python Server	nginx
Requests/sec – Run 1	4567.40	11126.44
Requests/sec – Run 2	4727.62	10178.01
Requests/sec – Run 3	4799.27	12428.07
Average requests/sec	4698.10	11244.17
Average time/request	2.129 ms	0.896 ms
Failed requests	0	0

Performance Result

nginx achieved approximately $2.39\times$ higher throughput

Both servers successfully processed all benchmark requests without failures.

Why is nginx Faster?

Custom Python Server

~4,698 req/s

- **Python implementation**
Higher runtime overhead
- **Thread-per-connection**
A separate thread handles each client connection
- **Simple file handling**
Files are opened and read for each request
- **Educational HTTP parser**
Designed for simplicity and protocol demonstration

nginx

~11,244 req/s

- **C implementation**
Low-level and highly optimized
- **Event-driven architecture**
Workers efficiently handle many connections
- **Optimized static file serving**
Designed specifically for efficient web serving
- **Production-grade implementation**
Mature and extensively optimized

Conclusion

Implemented

- Minimal HTTP/1.1 server using TCP sockets
- Manual HTTP request and response processing
- Static file serving and HTTP status codes
- Concurrent client handling using threads
- HTTPS support using TLS
- Linux deployment using Docker and Railway

Evaluated

- HTTP and HTTPS traffic using Wireshark
- TLS encryption in practice
- Custom server vs nginx
- Performance using ApacheBench
- Multiple benchmark runs in the same cloud environment

Final Result

A functional HTTP/HTTPS server was implemented from scratch, deployed in a Linux environment, and experimentally evaluated.

The custom server achieved approximately **4,698 requests/sec**, while nginx achieved **11,244 requests/sec** under the same benchmark conditions.

Thank You for Your Attention

Questions?

Source Code

`github.com/ErnestMatskevich/minimal-http-server`

Ernest Matskevich